OBSERVABLES Y SIGNALS: RXJS Y REACTIVIDAD EN ANGULAR

RxJS, observables, observers, suscripciones, subjects (Subject, BehaviorSubject, ReplaySubject, AsyncSubject), operadores, Angular Signals y comparativa Signals vs. observables

Apuntes revisados y ampliados

Documento elaborado a partir del fastbook original de Rodrigo Zavala

Contenido

1. RxJS . . . 3
1.1 Qué es RxJS . . . 3
1.2 Observadores . . . 3
1.3 Operadores, subjects y suscripciones . . . 3
2. ¿Qué es un observable? . . . 4
2.1 Creación de observables . . . 4
2.2 Características clave . . . 5
2.3 Suscripciones . . . 5
3. Observers . . . 5
3.1 Definición e implementación . . . 5
3.2 Los tres métodos . . . 6
3.3 Suscripción y desuscripción . . . 6
3.4 Flujo de datos y reactividad . . . 6
4. Tipos de observables: subjects . . . 6
4.1 Subject básico . . . 6
4.2 BehaviorSubject . . . 7
4.3 ReplaySubject . . . 7
4.4 AsyncSubject . . . 8
5. Operadores . . . 9
5.1 Operadores de creación . . . 9
5.2 Operadores de transformación . . . 9
5.3 Operadores de filtrado . . . 10
6. Signals . . . 10
6.1 Qué son . . . 10
6.2 Ejemplo: lista de tés con precio total . . . 11
6.3 Interoperabilidad con observables . . . 11
7. Conclusiones: Signals vs. observables . . . 12
8. Qué hemos aprendido . . . 13

Introducción

Gestionar bien los datos asíncronos (peticiones HTTP, eventos del usuario, temporizadores…) es clave en cualquier aplicación moderna. Para ello Angular se apoya en **RxJS**, la librería de programación reactiva para JavaScript.

El tema recorre RxJS de abajo arriba:

- **Observables:** el corazón de RxJS; representan flujos de datos y eventos.
- **Observers y suscripciones:** cómo se reciben y se dejan de recibir esos valores.
- **Subjects:** observables especiales que reparten valores a varios observadores.
- **Operadores:** funciones para crear, transformar y filtrar flujos.
- **Angular Signals:** la nueva forma de reactividad del framework, en qué se diferencia de los observables y hacia dónde va Angular.

1. RxJS

1.1 Qué es RxJS

RxJS (*Reactive Extensions for JavaScript*) es una librería que implementa la **programación reactiva** en JavaScript: se centra en procesar **flujos de datos asíncronos o eventos**, lo que da lugar a código más limpio, sostenible y reutilizable en aplicaciones interactivas y complejas.

Su núcleo son los **observables**, que representan una colección de valores o eventos futuros. Un observable es una función que puede **enviar múltiples valores a lo largo del tiempo** y puede emitir tres tipos de notificación:

- **Un valor**.
- **Un error**.
- **Una notificación de finalización**.

Así, los observables permiten trabajar con colecciones de valores de forma generalizada y aplicar una amplia gama de operaciones sobre secuencias de datos.

1.2 Observadores

Para consumir estos flujos, RxJS proporciona **observadores**: objetos que implementan tres métodos.

Método	Para qué sirve
next	Recibir el próximo valor emitido por el observable.
error	Recibir un error emitido.
complete	Recibir la notificación de que no se emitirán más valores.

1.3 Operadores, subjects y suscripciones

- **Operadores:** funciones puras que componen observables mediante operaciones como *map*, *filter* o *concat*. Permiten construir tuberías de datos donde los valores se transforman, filtran y manipulan de forma declarativa.

- **Subjects (sujetos):** actúan a la vez como observador y observable. Mantienen una lista de observadores y emiten valores a todos simultáneamente (*multicasting*: un observable compartido entre varios observadores).
- **Suscripción:** representa la **ejecución** de un observable. Sirve para iniciar la observación y para **liberar los recursos** cuando ya no es necesaria. Al suscribirse, el observable empieza a emitir y los valores los manejan los métodos del observador.

RxJS permite crear aplicaciones más robustas y eficientes gestionando flujos asíncronos con facilidad, y encaja especialmente bien en **SPA** con frameworks como Angular.

> **★ Lo más importante**
>
> ► **RxJS** = Reactive Extensions for JavaScript · programación reactiva sobre flujos asíncronos.
>
> ► Un **observable** emite múltiples valores en el tiempo y tres tipos de notificación: **valor, error, finalización**.
>
> ► Un **observador** implementa **next**, **error** y **complete**.
>
> ► **Operadores** = funciones puras para componer flujos · **subjects** = observable + observador (multicasting) · **suscripción** = ejecución del observable y liberación de recursos.

2. ¿Qué es un observable?

Un observable es una **abstracción que representa una fuente de valores o eventos** que pueden emitirse en cualquier momento del tiempo. Es el tipo de objeto central de la programación reactiva con RxJS.

2.1 Creación de observables

Se crean con la clase **Observable** de RxJS, bien instanciándola y pasándole una función que define su comportamiento, bien con funciones auxiliares como *of*, *from* o *interval*.

```
import { Observable } from 'rxjs';

const observable = new Observable(observer => {
  observer.next('Hello');
  observer.next('World');
  observer.complete();
});
```

2.2 Características clave

Característica	Qué significa
Producción de valores	Produce una secuencia de valores de forma **asíncrona**: puede emitir muchos durante su vida, no un único valor de retorno como una función.
Flujo unidireccional	Los observadores reciben valores, errores o la finalización, pero **no pueden enviar datos de vuelta** al observable.
Lazy evaluation	Es **perezoso**: no emite nada hasta que alguien se suscribe. Da más control sobre el flujo y operaciones eficientes en memoria.
Operadores	No son parte del observable, pero son clave: reciben un observable y **devuelven uno nuevo**, permitiendo flujos complejos.
Finalización	Al completarse ya no emite más valores; permite **liberar recursos** y que los observadores reaccionen al fin de la operación.

2.3 Suscripciones

Para recibir los valores, un observador debe **suscribirse** con el método *subscribe*, que acepta un objeto observador con los métodos *next* (valores), *error* (errores) y *complete* (finalización).

```
observable.subscribe({
  next: value => console.log(value),
  error: err => console.error(err),
  complete: () => console.log('Completed')
});
```

En resumen: un observable es una fuente potente y flexible de valores o eventos que facilita la programación asíncrona y reactiva y permite manejar flujos de datos de forma eficiente y estructurada.

★ Lo más importante

► Se crea con **new Observable(observer => {...})** o con funciones como **of**, **from**, **interval**.
► Emite **varios valores asíncronos** · flujo **unidireccional** (el observador no devuelve datos).
► Es **lazy**: no emite nada hasta que hay una suscripción.
► **subscribe({ next, error, complete })** inicia la recepción de valores.

3. Observers

3.1 Definición e implementación

Un *observer* es la entidad que interpreta las señales de un observable: una interfaz que dicta cómo reaccionar a cada tipo de notificación. En la práctica es un **objeto con tres manejadores de callback**: *next*, *error* y *complete*.

```
const myObserver = {
  next: x => console.log('Observer got a next value: ' + x),
  error: err => console.error('Observer got an error: ' + err),
  complete: () => console.log('Observer got a complete notification'),
};
```

3.2 Los tres métodos

Método	Cuándo se ejecuta y para qué sirve
next	El **más utilizado**. Se llama cada vez que el observable emite un valor, para reaccionar a cada emisión individualmente.
error	Se invoca cuando ocurre un error durante la ejecución del observable; permite capturarlo y manejarlo de forma ordenada y predecible.
complete	Se activa cuando el observable termina de emitir; indica que la secuencia concluyó bien y permite ejecutar lógica de limpieza o cierre.

3.3 Suscripción y desuscripción

El observer se vincula al observable con **subscribe**, estableciendo un canal por el que recibe las notificaciones según los métodos que haya definido.

```
myObservable.subscribe(myObserver);
```

La suscripción puede terminarse con **unsubscribe**, de modo que el observer deja de observar. Es vital para **prevenir fugas de memoria** y liberar recursos que ya no se necesitan.

Nota práctica (no aparece en el fastbook): subscribe() devuelve un objeto Subscription; se guarda (const sub = obs$.subscribe(...)) para llamar después a sub.unsubscribe(), normalmente en ngOnDestroy. En Angular también se usan el pipe async o takeUntilDestroyed() para que la desuscripción sea automática.

3.4 Flujo de datos y reactividad

El observer es la pieza clave del patrón *observer* en RxJS: permite **reaccionar dinámicamente a los cambios** de datos o estado representados por los observables, dando lugar a aplicaciones más responsivas, robustas, mantenibles y escalables.

> **★ Lo más importante**
> ► Un **observer** es un objeto con los callbacks **next**, **error** y **complete**.
> ► **next** = cada valor (el más usado) · **error** = fallo · **complete** = fin de la secuencia.
> ► **subscribe(observer)** conecta · **unsubscribe()** desconecta y evita **fugas de memoria**.

4. Tipos de observables: subjects

Los *subjects* son un **puente entre observables y observadores**: permiten que varios observadores reciban las notificaciones de una única fuente. RxJS ofrece cuatro tipos.

4.1 Subject básico

Actúa como **proxy**: es a la vez observador y observable y hace **multicasting**, de modo que todos los observadores ven los mismos valores. Solo reciben los valores emitidos **después** de suscribirse.

```
import { Subject } from 'rxjs';

const subject = new Subject();

subject.subscribe({
  next: (v) => console.log(`observerA: ${v}`)
});

subject.subscribe({
  next: (v) => console.log(`observerB: ${v}`)
});

subject.next(1);
subject.next(2);
```

```
// Salida por consola
observerA: 1
observerB: 1
observerA: 2
observerB: 2
```

4.2 BehaviorSubject

Variante que **almacena el último valor emitido** y necesita un **valor inicial**. Quien se suscribe recibe inmediatamente ese último valor, lo que equivale a tener un **estado inicial**.

```
import { BehaviorSubject } from 'rxjs';

const behaviorSubject = new BehaviorSubject(0);

behaviorSubject.subscribe({
  next: (v) => console.log(`observerA: ${v}`)
});

behaviorSubject.next(1);
behaviorSubject.next(2);

behaviorSubject.subscribe({
  next: (v) => console.log(`observerB: ${v}`)
});
```

```
// Salida por consola
observerA: 0
observerA: 1
observerA: 2
observerB: 2   // B recibe al suscribirse el último valor (2)
```

Nota práctica (no aparece en el fastbook): en el fastbook los dos bloques de código anteriores aparecen intercambiados (el código del BehaviorSubject bajo «Subject básico» y viceversa); aquí se muestran emparejados con su salida correcta.

4.3 ReplaySubject

Como el BehaviorSubject, envía valores antiguos a los nuevos suscriptores, pero puede **registrar y reemitir varios**, no solo el último. El número de valores a retransmitir se indica **como argumento al instanciarlo**.

```
import { ReplaySubject } from 'rxjs';

const replaySubject = new ReplaySubject(2);  // guarda los 2 últimos valores

replaySubject.next(1);
replaySubject.next(2);
replaySubject.next(3);

replaySubject.subscribe({
  next: (v) => console.log(`observerA: ${v}`)
});

// Salida por consola
// observerA: 2
// observerA: 3
```

4.4 AsyncSubject

Envía **solo el último valor** y **únicamente cuando el observable se completa**: espera al *complete()* antes de emitir.

```
import { AsyncSubject } from 'rxjs';

const asyncSubject = new AsyncSubject();

asyncSubject.subscribe({
  next: (v) => console.log(`observerA: ${v}`)
});

asyncSubject.next(1);
asyncSubject.next(2);
asyncSubject.complete();

asyncSubject.subscribe({
  next: (v) => console.log(`observerB: ${v}`)
});

// Salida por consola
// observerA: 2
// observerB: 2
```

Tipo	Qué recibe un nuevo suscriptor
Subject	Solo los valores emitidos después de suscribirse.
BehaviorSubject	El último valor (o el inicial) al momento, y luego los nuevos. Requiere valor inicial.
ReplaySubject(n)	Los n últimos valores al momento, y luego los nuevos.
AsyncSubject	Solo el último valor y solo cuando se llama a complete().

Cada tipo tiene su uso según el comportamiento que se quiera en el flujo de datos; conocer sus diferencias permite elegir el adecuado.

> **★ Lo más importante**
> ► **Subject** = multicasting · solo valores futuros.
> ► **BehaviorSubject(inicial)** = guarda el **último** valor → ideal para estado.
> ► **ReplaySubject(n)** = reemite los **n últimos** valores.
> ► **AsyncSubject** = emite solo el **último** valor y solo tras **complete()**.

5. Operadores

Los operadores son **funciones puras** que crean nuevos observables a partir de otros, para manipular y procesar flujos de datos. Se encadenan con **pipe()**.

5.1 Operadores de creación

Crean observables **desde cero** o a partir de otras fuentes de datos.

- **of:** crea un observable que emite los valores indicados y después se completa.

```
import { of } from 'rxjs';

of(1, 2, 3).subscribe(val => console.log(val));
// Salida: 1, 2, 3
```

- **from:** crea un observable a partir de un *array*, una *promise*, un iterable o colección similar.

```
import { from } from 'rxjs';

from([1, 2, 3]).subscribe(val => console.log(val));
// Salida: 1, 2, 3
```

5.2 Operadores de transformación

Modifican los valores emitidos por un observable.

- **map:** aplica una función a cada valor emitido y emite el resultado transformado.

```
import { of } from 'rxjs';
import { map } from 'rxjs/operators';

of(1, 2, 3)
  .pipe(map(x => x * 10))
  .subscribe(val => console.log(val));
// Salida: 10, 20, 30
```

- **scan:** como *reduce* de un array: aplica una función a cada valor junto al acumulado y **emite cada resultado acumulado**.

```
import { of } from 'rxjs';
import { scan } from 'rxjs/operators';

of(1, 2, 3)
  .pipe(scan((acc, curr) => acc + curr, 0))
  .subscribe(val => console.log(val));
// Salida: 1, 3, 6
```

5.3 Operadores de filtrado

Seleccionan qué valores se emiten según ciertos criterios.

- **filter:** solo deja pasar los valores que cumplen una condición.

```
import { from } from 'rxjs';
import { filter } from 'rxjs/operators';

from([1, 2, 3, 4])
  .pipe(filter(x => x % 2 === 0))
  .subscribe(val => console.log(val));
// Salida: 2, 4
```

- **take:** solo deja pasar los **n primeros** valores y después se completa.

```
import { from } from 'rxjs';
import { take } from 'rxjs/operators';

from([1, 2, 3, 4])
  .pipe(take(2))
  .subscribe(val => console.log(val));
// Salida: 1, 2
```

Operador	Tipo · qué hace
of	Creación · emite los valores dados y completa.
from	Creación · convierte array, promise o iterable en observable.
map	Transformación · transforma cada valor.
scan	Transformación · acumula y emite cada resultado parcial.
filter	Filtrado · deja pasar los que cumplen la condición.
take(n)	Filtrado · deja pasar los n primeros y completa.

★ Lo más importante

► Operadores = **funciones puras** que devuelven un **nuevo observable**; se encadenan con **.pipe()**.

► Creación: **of** (valores sueltos) · **from** (array, promise, iterable).

► Transformación: **map** (cada valor) · **scan** (acumulado, como reduce pero emitiendo cada paso).

► Filtrado: **filter** (condición) · **take(n)** (primeros n y completa).

6. Signals

6.1 Qué son

Los **Signals** son la nueva forma de entender la reactividad en Angular, incluidos a partir de **Angular 16**. Crean **relaciones reactivas entre datos**: cuando el valor de uno cambia, todo lo que depende de él se notifica y se actualiza automáticamente.

Son nuevos en Angular, pero llevan tiempo usándose en **otros frameworks**, así que lo aprendido es transversal.

6.2 Ejemplo: lista de tés con precio total

El componente tiene:

- **teas:** un array de tés con nombre y precio.
- **itemList:** un Signal creado con *signal()* a partir del array.
- **totalPrice:** un *computed* que, con un *reduce*, calcula el precio total a partir del Signal.
- **removeItem:** una función que elimina un elemento con *set()*.

Al eliminar un elemento se actualiza el Signal de la lista y, en consecuencia, la computada que depende de él se notifica y se recalcula.

```
import { CommonModule } from '@angular/common';
import { Component, computed, signal } from '@angular/core';

@Component({
  selector: 'example-component',
  imports: [CommonModule],
  standalone: true,
  template: `
    <ul>
      <li *ngFor="let item of teas">
        {{item.name}} - {{item.price}}$
        <button (click)="removeItem(item)">Remove</button>
      </li>
    </ul>
    Total Price: {{totalPrice()}}$
  `,
})
export class ExampleComponent {
  teas = [
    { name: 'A', price: 10 },
    { name: 'B', price: 15 },
    { name: 'C', price: 20 },
  ];

  itemList = signal(this.teas);

  totalPrice = computed(() => {
    return this.itemList().reduce((acc, curr) => acc + curr.price, 0);
  });

  removeItem(item: any) {
    this.itemList.set(this.itemList().filter((i) => i !== item));
  }
}
```

Analizando el ejemplo:

- **signal(valor):** crea un Signal; su valor se **lee llamándolo como función**: *itemList()*.
- **set(nuevoValor):** reemplaza el valor y notifica a quien dependa de él.
- **computed(() => ...):** valor derivado que se recalcula solo cuando cambian los Signals que usa; en la plantilla se lee como *totalPrice()*.

6.3 Interoperabilidad con observables

Angular permite convertir entre ambos mundos: **toObservable()** pasa de Signal a observable y **toSignal()** de observable a Signal.

```
export class ExampleComponent {
  value = signal('');
  value$ = toObservable(this.value);   // Signal -> observable

  ngOnInit() {
    this.value$.subscribe(console.log);
  }
}
```

```
export class ExampleComponent {
  value$ = from([1, 2]);
  value = toSignal(this.value$);        // observable -> Signal
}
```

*Nota práctica (no aparece en el fastbook): toObservable y toSignal se importan desde '@angular/core/rxjs-interop'. Además de set(), los Signals tienen update(fn) para calcular el nuevo valor a partir del anterior. En Angular 17+ el ejemplo suele escribirse con @for en lugar de *ngFor, y en Angular 19+ los componentes son standalone por defecto.*

★ Lo más importante

► **Signals** (Angular 16+) = reactividad nativa: al cambiar un valor se actualiza todo lo que depende de él.

► **signal(v)** crea · **s()** lee · **s.set(v)** modifica.

► **computed(() => ...)** = valor derivado que se recalcula automáticamente.

► **toObservable()**: Signal → observable · **toSignal()**: observable → Signal.

7. Conclusiones: Signals vs. observables

- **Más sencillos:** los Signals son mucho más fáciles de entender que los observables y están en el **core de Angular**, lo que reduce la dependencia de RxJS, una librería potente pero compleja que ha dado a Angular fama de difícil.
- **Sin suscripciones:** una de sus mayores ventajas es que no hay que gestionar suscripciones, que en aplicaciones complejas pueden provocar **fugas de memoria** si no se manejan bien.
- **Tendencia:** la apuesta de Angular por los Signals lo acerca a React, Vue.js y otros frameworks, buscando ser más amigable para el desarrollador y lo más nativo posible.

Aspecto	Observables (RxJS) · Signals
Origen	Librería externa RxJS · Core de Angular
Complejidad	Mayor curva de aprendizaje · Más sencillos
Lectura del valor	subscribe() · Llamar al Signal: s()
Suscripciones	Hay que gestionarlas (unsubscribe) · No hay
Derivados	Operadores (map, filter...) · computed()
Conversión	toSignal() · toObservable()

★ Lo más importante

► Signals: **más simples**, nativos de Angular y **sin suscripciones** que gestionar.
► Observables siguen siendo útiles para flujos asíncronos complejos y operadores.
► Ambos conviven: **toSignal** / **toObservable**.
► Angular se mueve hacia los Signals, en la línea de React y Vue.

8. Qué hemos aprendido

Hemos visto que **RxJS** aporta programación reactiva a JavaScript mediante **observables**: fuentes perezosas y unidireccionales que emiten valores, errores o la finalización a lo largo del tiempo. Los **observers** consumen esas emisiones con *next*, *error* y *complete*, se conectan con *subscribe* y deben desconectarse con *unsubscribe* para evitar fugas de memoria. Conocimos los cuatro **subjects** (Subject, BehaviorSubject, ReplaySubject y AsyncSubject), que reparten valores a varios observadores con distinto comportamiento, y los **operadores** básicos de creación (*of*, *from*), transformación (*map*, *scan*) y filtrado (*filter*, *take*). Por último, estudiamos los **Angular Signals** (*signal*, *set*, *computed*, *toObservable*, *toSignal*) y por qué representan el futuro de la reactividad en Angular: son más sencillos y eliminan la gestión de suscripciones. El siguiente paso es aplicar todo esto en servicios y componentes reales, combinando Signals para el estado y RxJS para los flujos asíncronos.

¡Enhorabuena! Documento completado